ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

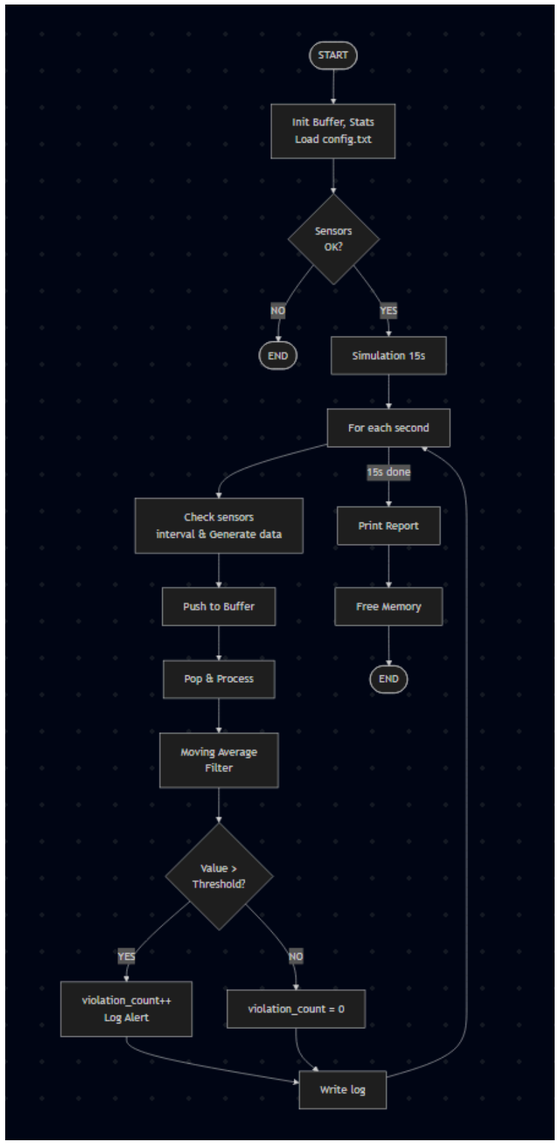
Hệ Thống Giám Sát Môi Trường Trong Nhà

Trình bày :

Phạm Đức Duy : Readme
Mai Bảo Đăng : Lập Trình
Ngô Đức Long : Slide + Flowchart
Nguyễn Lâm Duy : Lập Trình

ONE LOVE. ONE FUTURE.

Chi tiết flowchart



datatypes.h

Hằng số & Cấu hình thiết bị | datatypes.h

```c
#define MAX_SENSORS 30
#define MAX_STRING_LEN 50
#define BUFFER_CAPACITY 100        // Dung lượng bộ đệm

// 1. CẤU TRÚC DỮ LIỆU
// 1.1 Struct cấu hình thiết bị
typedef struct {
    int id;
    char type[MAX_STRING_LEN];      // Loại cảm biến
    char location[MAX_STRING_LEN];  // Vị trí lắp đặt
    float threshold;                // Ngưỡng cảnh báo
    int send_interval;              // Chu kỳ gửi dữ liệu
} SensorSet;

// 1.2 Struct bản tin
typedef struct {
    int sensor_id;
    time_t timestamp;
    float value;
} SensorMessage;
```

- Hằng số: quy định giới hạn hệ thống (số cảm biến, độ dài chuỗi, dung lượng bộ đệm).
- SensorSet: cấu hình từng thiết bị, giúp hệ thống biết cách giám sát.
- SensorMessage: bản tin dữ liệu đo được, gắn với thời gian thực.
- Vai trò: đảm bảo dữ liệu từ cảm biến được chuẩn hóa trước khi xử lý.

Bộ đệm & Thống kê lỗi | datatypes.h

```c
// 1.3 Struct bộ đệm
typedef struct {
    SensorMessage *messages; // Mảng chứa các bản tin
    int capacity;            // Sức chứa tối đa
    int head;                // Vị trí ghi tiếp theo
    int tail;                // Vị trí đọc tiếp theo
    int count;               // Số lượng bản tin đang có trong bộ đệm
} Buffer;

// 1.4 Struct thống kê lỗi
typedef struct {
    int valid_msg_count;        //Số lượng bản tin hợp lệ
    int error_msg_count;        //Số lượng bản tin lỗi
    int threshold_exceed_count; //Số lần vượt ngưỡng
    int buffer_overflow_count;  //Số lần tràn bộ đệm
    int missed_msg_count;       //Số lượng bản tin bị bỏ qua
} Stats;
```

Buffer: quản lý luồng bản tin theo cơ chế vòng (FIFO), tránh mất dữ liệu.

Stats: theo dõi hiệu năng, phát hiện sự cố (lỗi, vượt ngưỡng, tràn bộ đệm, bỏ sót).

Mối liên hệ:
SensorSet → tạo bản tin (SensorMessage).

SensorMessage → lưu vào Buffer.

Buffer → được giám sát bởi Stats để đánh giá chất lượng hệ thống.

Ứng dụng thực tế: đảm bảo dữ liệu cảm biến được xử lý chính xác.

HUST

func.c & func.h

Khởi tạo & Đọc cấu hình | func.c

```c
16  void initBuffer(Buffer *b, int capacity) {
17      b->capacity = capacity;
18      b->messages = (SensorMessage*)malloc(capacity * sizeof(SensorMessage));
19      b->head = 0;
20      b->tail = 0;
21      b->count = 0;
22  }
23
24  // Hàm giải phóng bộ đệm
25  void freeBuffer(Buffer *b) {
26      if (b->messages != NULL) {
27          free(b->messages);
28      }
29  }
30
31  // Hàm set thống kê lỗi về 0
32  void initStats(Stats *stats) {
33      memset(stats, 0, sizeof(Stats));
34  }
35
36  // Hàm đọc file
37  int loadFile(const char* filename, SensorSet sensors[]) {
38      FILE *file = fopen(filename, "r");
39      if (!file) {
40          printf("Khong the mo file %s. Vui long kiem tra lai!\n", filename);
41          return 0; // Trả về 0 thiết bị
42      }
43
44      char line[100];
45      int count = 0;
46
47      // Đọc và bỏ qua dòng đầu của file txt
48      fgets(line, sizeof(line), file);
49
50      // Đọc từng dòng cho đến hết file hoặc đạt giới hạn MAX_SENSORS
51      while (fgets(line, sizeof(line), file) && count < MAX_SENSORS) {
52          if (sscanf(line, "%d,%[^,],%[^,],%f,%d",
53                     &sensors[count].id,
54                     sensors[count].type,
55                     sensors[count].location,
56                     &sensors[count].threshold,
57                     &sensors[count].send_interval) == 5) {
58
59              // Khởi tạo bộ nhớ cho filter
60              memset(sensors[count].filter_history, 0, sizeof(sensors[count].filter_history));
61              sensors[count].filter_index = 0;
62              sensors[count].violation_count = 0;
63
64              count++;
65          }
66      }
67
68      fclose(file);
69      return count; // Trả về số lượng thiết bị đọc được
70  }
```

Hàm	Dòng	Chức năng	Input	Output	Ghi chú
initBuffer()	16-22	Cấp phát bộ nhớ buffer vòng	capacity (int)	Không	Khởi tạo head=0, tail=0, count=0
freeBuffer()	25-29	Giải phóng bộ nhớ buffer	Buffer *b	Không	Tránh rò rỉ bộ nhớ
initStats()	32-34	Đặt lại thống kê về 0	Stats *stats	Không	Dùng memset()
loadFile()	37-70	Đọc file cấu hình CSV	filename (char*), sensors[]	int (số thiết bị)	Phân tích 5 trường: ID, Type, Location, Threshold, Interval

Mô Phỏng Producer-Consumer | func.c

```c
99  //3. MÔ PHỎNG SINH DỮ LIỆU
100 float generateRandomVal(const char* type) {
101     if (strcmp(type, "TEMP") == 0) return 20.0 + (rand() % 300) / 10.0;
102     if (strcmp(type, "HUMID") == 0) return 40.0 + (rand() % 500) / 10.0;
103     if (strcmp(type, "GAS") == 0) return (rand() % 1000) / 10.0;
104     if (strcmp(type, "DUST") == 0) return (rand() % 2000) / 10.0;
105     return (rand() % 100);
106 }
107
108 void runSimulation(SensorSet sensors[], int num_sensors, Stats *stats, Buffer *b, int duration) {
109     printf("\n    BAT DAU MO PHONG TRONG %d GIAY    \n", duration);
110     writeLog("BAT DAU MO PHONG");
111
112     for (int t = 1; t <= duration; t++) {
113         //  -- NHÁNH 1: PRODUCER
114         for (int i = 0; i < num_sensors; i++) {
115             if (t % sensors[i].send_interval == 0) {
116                 // 1.Tạo bản tin
117                 SensorMessage newMsg;
118                 newMsg.sensor_id = sensors[i].id;
119                 newMsg.value = generateRandomVal(sensors[i].type);
120                 newMsg.timestamp = time(NULL);
121
122                 pushBuffer(b, newMsg, stats);      // đẩy vào buffer luôn
123
124                 // In ra màn hình
125                 printf("[Giay %2d] Sensor (ID: %d) -> GUI DU LIEU: %.2f\n", t, newMsg.sensor_id, newMsg.value);
126                 stats->valid_msg_count++;
127             }
128         }
129         // -- NHÁNH 2: COMSUMER
130         SensorMessage procMsg;
131         while (popBuffer(b, &procMsg)) {
132         // tìm đúng sensor để lấy thông tin
133            for (int k = 0; k < num_sensors; k++) {
134                 if (sensors[k].id == procMsg.sensor_id) {
135
136                     float filtered_val = applyMovingAverage(&sensors[k], procMsg.value);    // Lọc nhiễu (Dùng giá trị lấy từ Buffer)
137
138                     handleThreshold(&sensors[k], filtered_val, stats);      // Kiểm tra ngưỡng và Cảnh báo
139
140                     // Ghi log sự kiện
141                     char log_msg[150];
142                     sprintf(log_msg, "Da xu ly tu Buffer - ID: %d, Gia tri loc: %.2f", procMsg.sensor_id, filtered_val);
143                     writeLog(log_msg);
144
145                     break; // Xử lý xong 1 bản tin, chuyển sang bản tin tiếp theo trong buffer
146                 }
147             }
148         }
149     }
150
151     writeLog("KET THUC MO PHONG");
152     printf("       KET THUC MO PHONG    \n");
153 }
```

Hàm	Dòng	Chức năng	Input	Output	Ghi chú
generateRandomVal()	100-106	Sinh dữ liệu ngẫu nhiên theo loại cảm biến	type (char*)	float	TEMP: 20-50°C, HUMID: 40-90%, GAS: 0-100 ppm, DUST: 0-200 µg/m³
runSimulation()	108-153	Chạy vòng lặp mô phỏng Producer-Consumer	sensors[], num_sensors, stats*, buffer*, duration	Không	**PRODUCER**: Sinh dữ liệu theo send_interval, đẩy buffer **CONSUMER**: Lấy buffer, lọc nhiễu, kiểm tra ngưỡng, ghi log

Lọc Nhiễu & Phát Hiện Cảnh Báo | func.c

```c
158 float applyMovingAverage(SensorSet *s, float newVal) {
159     // để tránh bị các số 0 kéo tụt trung bình xuống.
160     if (s->filter_index == 0 && s->filter_history[1] == 0.0f) {
161         for (int i = 0; i < FILTER_SIZE; i++) {
162             s->filter_history[i] = newVal;
163         }
164     }
165
166     s->filter_history[s->filter_index] = newVal;            // lưu mẫu mới vào lịch sử
167     s->filter_index = (s->filter_index + 1) % FILTER_SIZE;
168
169     // tính trung bình cộng 5 mẫu gần nhất
170     float sum = 0;
171     for (int i = 0; i < FILTER_SIZE; i++) {
172         sum += s->filter_history[i];
173     }
174
175     return sum / (float)FILTER_SIZE;
176 }
177
178 // Hàm vượt ngưỡng
179 void handleThreshold(SensorSet *s, float filteredVal, Stats *stats) {
180     if (filteredVal > s->threshold) {
181          s->violation_count++;     // tăng bộ đếm vi phạm liên tiếp
182
183     // Trường hợp 1: vượt ngưỡng tức thì
184         if (s->violation_count == 1) {
185             printf("    [CANH BAO] Sensor (ID: %d): Chom vuot nguong (%.2f). Dang theo doi...\n", s->id, filteredVal);
186         }
187
188     // Trường hợp 2: vượt ngưỡng kéo dài
189          else if (s->violation_count >= 2) {
190             stats->threshold_exceed_count++;      // Chỉ ghi nhận vào thống kê khi đã xác nhận sự cố kéo dài
191
192             char alert_msg[150];
193             sprintf(alert_msg, "SU CO KEO DAI - Sensor (ID: %d), Gia tri loc: %.2f (Threshold: %.2f)",
194                     s->id, filteredVal, s->threshold);
195             writeLog(alert_msg);
196             printf("   [!] NGUY HIEM: %s\n", alert_msg);
197         }
198     }
199     else {
200         // Nếu giá trị quay về mức an toàn: Reset bộ đếm
201         if (s->violation_count >= 2) {
202             printf("   [INFO] Sensor %d: Da on dinh tro lai.\n", s->id);
203             }
204         s->violation_count = 0;
205         }
206 }
```

Hàm	Dòng	Chức năng	Input	Output	Ghi chú
applyMovingAverage()	158-176	Lọc nhiễu bằng Moving Average Filter	SensorSet *s, newVal (float)	float (giá trị lọc)	Giữ FILTER_SIZE mẫu → Tính TB Quay vòng với filter_index Loại bỏ spike nhiễu đơn lẻ
handleThreshold()	179-206	Phát hiện cảnh báo 2 cấp độ	SensorSet *s, filteredVal (float), Stats *stats	Không	**CẤP 1** (lần 1): [CANH BAO] - Đang theo dõi **CẤP 2** (lần 2+): [NGUY HIEM] - Sự cố kéo dài Reset violation_count khi quay bình thường

Quản Lý Bộ Đệm & Logging | func.c

```c
// ========== GHI DỮ LIỆU VÀO BUFFER (FIFO Vòng) ==========
void pushBuffer(Buffer *b, SensorMessage msg, Stats *stats) {
    b->messages[b->head] = msg;
    b->head = (b->head + 1) % b->capacity;     // Quay vòng khi vượt capacity

    if (b->count < b->capacity) {
        b->count++;  // Chưa đầy: tăng counter
    } else {
        b->tail = (b->tail + 1) % b->capacity;  // Đầy: ghi đè dữ liệu cũ
        stats->buffer_overflow_count++;
        writeLog("WARNING: Buffer Overflow! Oldest message overwritten.");
    }
}

// ========== ĐỌC DỮ LIỆU TỪ BUFFER (FIFO) ==========
int popBuffer(Buffer *b, SensorMessage *outMsg) {
    if (b->count == 0) return 0;  // Rỗng

    *outMsg = b->messages[b->tail];            // Lấy từ tail (cũ nhất)
    b->tail = (b->tail + 1) % b->capacity;     // Quay vòng
    b->count--;
    return 1;  // Thành công
}

// ========== GHI LOG SỰ KIỆN VỚI TIMESTAMP ==========
void writeLog(const char* message) {
    FILE *f = fopen("report.log", "a");  // Chế độ append
    if (f == NULL) return;

    time_t now = time(NULL);
    char *timestamp = ctime(&now);
    timestamp[strlen(timestamp) - 1] = '\0';  // Xóa ký tự xuống dòng

    fprintf(f, "[%s] %s\n", timestamp, message);
    fclose(f);
}

// ========== XUẤT BÁO CÁO TỔNG HỢP ==========
void printReport(Stats stats) {
    printf("\n----------------------------------------\n");
    printf("        BAO CAO TONG HOP HE THONG         \n");
    printf("----------------------------------------\n");
    printf("- So ban tin hop le: %d\n", stats.valid_msg_count);
    printf("- So ban tin bi loi: %d\n", stats.error_msg_count);
    printf("- So lan vuot nguong: %d\n", stats.threshold_exceed_count);
    printf("- So lan tran bo dem: %d\n", stats.buffer_overflow_count);
    printf("- So ban tin bi bo qua: %d\n", stats.missed_msg_count);
    printf("----------------------------------------\n");
}
```

Hàm	Dòng	Chức năng	Input	Output	Ghi chú
pushBuffer()	211-225	Ghi dữ liệu vào buffer FIFO vòng	Buffer *b, SensorMessage msg, Stats *stats	Không	Head quay vòng: (head+1)%capacity Chưa đầy: count++ Đầy: tail++ (ghi đè dữ liệu cũ)
popBuffer()	228-236	Đọc dữ liệu từ buffer FIFO	Buffer *b, SensorMessage *outMsg	int (0=rỗng, 1=thành công)	Tail quay vòng: (tail+1)%capacity FIFO: Lấy từ tail (cũ nhất) Giảm count
writeLog()	74-84	Ghi log sự kiện với timestamp	message (char*)	Không	File: report.log Chế độ append (ghi tiếp) Format: [TIMESTAMP] MESSAGE
printReport()	86-97	Xuất báo cáo tổng hợp	Stats stats	Không	5 chỉ số thống kê: ✓ Bản tin hợp lệ ✓ Bản tin lỗi ✓ Vượt ngưỡng ✓ Tràn buffer ✓ Bỏ qua

Function Headers | func.h

```c
#ifndef FUNC_H
#define FUNC_H

#include "datatypes.h"

//2
void initBuffer(Buffer *b, int capacity);
void freeBuffer(Buffer *b);
void initStats(Stats *stats);
int loadFile(const char* filename, SensorSet sensors[]);

//3 & 6
void writeLog(const char* message);
void printReport(Stats stats);
float generateRandomValue(const char* type);
void runSimulation(SensorSet sensors[], int num_sensors, Stats *stats, Buffer *b, int duration_seconds);

//4
float applyMovingAverage(SensorSet *s, float newVal);
void handleThreshold(SensorSet *s, float filteredVal, Stats *stats);

//5
void pushBuffer(Buffer *b, SensorMessage msg, Stats *stats);
int popBuffer(Buffer *b, SensorMessage *out_msg);
#endif
```

Mục	Hàm	Chức năng
Buffer & Khởi tạo	initBuffer, freeBuffer, initStats, loadFile	Khởi tạo các cấu trúc dữ liệu và tải cấu hình cảm biến
Ghi nhật ký & Mô phỏng (3 & 6)	writeLog, printReport, generateRandomValue, runSimulation	Ghi log, in báo cáo, tạo giá trị ngẫu nhiên, chạy mô phỏng chính
Xử lý Dữ liệu (4)	applyMovingAverage, handleThreshold	Lọc dữ liệu bằng trung bình động, xử lý vi phạm ngưỡng
Thao tác Hàng đợi (5)	pushBuffer, popBuffer	Thêm/lấy tin nhắn cảm biến từ bộ đệm

main.c

Khởi tạo & Đọc cấu hình | main.c

```c
15  int main() {
16      srand(time(NULL));
17
18      SensorSet sensors[MAX_SENSORS];
19      Buffer buffer;
20      Stats stats;
21
22      initBuffer(&buffer, BUFFER_CAPACITY);
23      initStats(&stats);
24
25      int num_sensors = loadFile("config.txt", sensors);
```

Chức Năng	Dòng Code	Mục Đích	Ghi Chú
Khởi tạo seed ngẫu nhiên	16	Mỗi lần chạy có dữ liệu khác nhau	srand(time(NULL))
Khai báo mảng cảm biến	18	Lưu tối đa 30 thiết bị	SensorSet sensors[MAX_SENSORS]
Khai báo bộ đệm	19	Lưu bản tin dữ liệu	Buffer buffer
Khai báo thống kê	20	Ghi nhận lỗi hệ thống	Stats stats
Khởi tạo bộ đệm	22	Cấp phát 100 ô nhớ	initBuffer(&buffer, 100)
Khởi tạo thống kê	23	Đặt tất cả counter = 0	initStats(&stats)
Tải file cấu hình	25	Đọc config.txt, nạp vào mảng	loadFile("config.txt", sensors)

Mô Phỏng & Báo cáo | main.c

```c
28      if (num_sensors > 0) {
29          printf("    HE THONG GIAM SAT MOI TRUONG    \n");
30          printf("File bao gom %d thiet bi\n\n", num_sensors);
31          printf("%-5s | %-10s | %-15s | %-10s | %-10s\n", "ID", "Type", "Location", "Threshold", "Interval");
32          printf("-----------------------------------------------------------------\n");
33
34          for (int i = 0; i < num_sensors; i++) {
35              printf("%-5d | %-10s | %-15s | %-10.2f | %-10d\n",
36                      sensors[i].id,
37                      sensors[i].type,
38                      sensors[i].location,
39                      sensors[i].threshold,
40                      sensors[i].send_interval);
41          }
42          //Chay mô phỏng trong 15s
43          runSimulation(sensors, num_sensors, &stats, &buffer, 15);
44          //In báo cáo
45          printReport(stats);
46
47      } else {
48          printf("Ko tim thay thiet bi.\n");
49      }
50
51      freeBuffer(&buffer);
52
53      return 0;
54  }
```

Chức Năng	Dòng Code	Mục Đích	Ghi Chú
Kiểm tra dữ liệu	28	Nếu num_sensors > 0 mới chạy	Tránh lỗi nếu thiếu cấu hình
In tiêu đề hệ thống	29	Hiển thị tên chương trình	Định dạng centered
In tiêu đề bảng	31-32	5 cột: ID | Type | Location | Threshold | Interval	Format căn lề (%-5s)
Vòng lặp in thiết bị	34-41	In từng cảm biến 1 dòng	Dùng printf với format chính xác
Chạy mô phỏng	43	Thu thập dữ liệu trong 15 giây	runSimulation(..., 15)
In báo cáo thống kê	45	Hiển thị kết quả lỗi & sự kiện	printReport(stats)
Giải phóng bộ nhớ	51	Tránh memory leak	freeBuffer(&buffer)

Kết Quả

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\JKM\adv-c\mini_project> ./main.exe
    HE THONG GIAM SAT MOI TRUONG
File bao gom 5 thiet bi

ID    | Type       | Location        | Threshold  | Interval
-----------------------------------------------------------------
1     | TEMP       | Phong_khach     | 35.50      | 5
2     | HUMID      | Phong_khach     | 80.00      | 5
3     | GAS        | Bep             | 50.00      | 2
4     | DUST       | Hanh_lang       | 100.00     | 10
5     | TEMP       | Phong_ngu       | 30.00      | 5

    BAT DAU MO PHONG TRONG 10 GIAY
[Giay  2] Sensor (ID: 3) -> GUI DU LIEU: 9.90
[Giay  4] Sensor (ID: 3) -> GUI DU LIEU: 15.80
[Giay  5] Sensor (ID: 1) -> GUI DU LIEU: 47.90
[Giay  5] Sensor (ID: 2) -> GUI DU LIEU: 80.90
[Giay  5] Sensor (ID: 5) -> GUI DU LIEU: 44.90
    [CANH BAO] Sensor (ID: 1): Chom vuot nguong (47.90). Dang theo doi...
    [CANH BAO] Sensor (ID: 2): Chom vuot nguong (80.90). Dang theo doi...
    [CANH BAO] Sensor (ID: 5): Chom vuot nguong (44.90). Dang theo doi...
[Giay  6] Sensor (ID: 3) -> GUI DU LIEU: 33.60
[Giay  8] Sensor (ID: 3) -> GUI DU LIEU: 67.10
[Giay 10] Sensor (ID: 1) -> GUI DU LIEU: 21.90
[Giay 10] Sensor (ID: 2) -> GUI DU LIEU: 74.50
[Giay 10] Sensor (ID: 3) -> GUI DU LIEU: 70.30
[Giay 10] Sensor (ID: 4) -> GUI DU LIEU: 21.00
[Giay 10] Sensor (ID: 5) -> GUI DU LIEU: 31.00
   [!] NGUY HIEM: SU CO KEO DAI - Sensor (ID: 1), Gia tri loc: 42.70 (Threshold: 35.50)
   [!] NGUY HIEM: SU CO KEO DAI - Sensor (ID: 5), Gia tri loc: 42.12 (Threshold: 30.00)
       KET THUC MO PHONG            BAO CAO TONG HOP HE THONG
----------------------------------------
- So ban tin hop le: 12
- So ban tin bi loi: 0
- So lan vuot nguong: 2
- So lan tran bo dem: 0
- So ban tin bi bo qua: 0
----------------------------------------
Chi tiet su kien da duoc luu tai file report.log
```

HUST

THANK YOU !